

Chat Canguçu

Aplicação de Chat em Tempo Real com WebSockets

Relatório Técnico — Disciplina de Redes de Computadores

Integrantes

Matheus Nunes Buttow

Luis Henrique Kiekhofel Reichow

Curso Engenharia da Computação

Instituição Universidade Federal do Rio Grande — FURG

Data Maio de 2026

Tecnologias Python 3 · WebSocket · Flask · JavaScript

Repositório github.com/matheusbuttow/chat-sockets-python

1. Motivação e Definição do Problema

O problema central abordado neste projeto é a ineficiência do modelo requisição-resposta do HTTP para comunicação em tempo real. Em aplicações de chat baseadas em polling, o cliente realiza requisições periódicas ao servidor independentemente de haver novas mensagens, gerando tráfego desnecessário e latência proporcional ao intervalo de consulta. O long polling reduz esse intervalo, mas mantém conexões TCP suspensas no servidor, limitando a escalabilidade.

O protocolo WebSocket (RFC 6455) resolve essa limitação estabelecendo um canal full-duplex sobre uma única conexão TCP. Após o handshake de upgrade via HTTP, ambas as partes podem transmitir frames a qualquer momento, com overhead de apenas 2 a 14 bytes por mensagem — contra centenas de bytes de cabeçalhos HTTP. O resultado é latência limitada apenas pela rede, sem round-trips adicionais de protocolo.

O Chat Canguçu — nome inspirado na cidade gaúcha de Canguçu — foi desenvolvido para demonstrar essa diferença na prática, implementando um sistema de mensagens com múltiplas salas, autenticação de usuários, histórico persistente e tolerância a falhas de servidor, utilizando dependências mínimas e sem brokers externos de mensageria.

2. Arquitetura do Sistema e Regras de Funcionamento

2.1 Visão Geral dos Serviços

A aplicação é composta por três processos independentes:

- **HTTP Server** (Flask, porta 5000): Serve os arquivos estáticos do frontend e expõe `/api/config`, que retorna as URLs dos servidores WebSocket via variáveis de ambiente — sem hardcode no cliente.
- **WebSocket Primário** (porta 8765): Servidor principal que gerencia conexões ativas, salas de chat, histórico de mensagens e autenticação de usuários.
- **WebSocket Backup** (porta 8766): Servidor de contingência que monitora o primário a cada 5 segundos. Após 3 falhas consecutivas, o frontend redireciona automaticamente para ele.

2.2 Diagrama de Arquitetura

WS Primário :8765	WS Backup :8766	HTTP Server :5000
Salas & Histórico Autenticação Thread por cliente	Monitora primário a cada 5 s Failover automático	index.html /api/config Assets estáticos

2.3 Modelo de Concorrência

Cada cliente recebe uma thread dedicada — `ClientThread` — que processa mensagens de forma assíncrona via `queue.Queue`. O callback `on_message()` apenas enfileira a mensagem; a thread consome e executa a lógica de negócio. Isso evita que um cliente lento bloqueie os demais.

Todo acesso ao estado global — dicionários de clientes, salas, histórico e usuários online — é protegido por `threading.Lock()`. O estado é centralizado em `server/globals.py`; os módulos especializados adquirem os locks apenas durante a janela crítica de leitura ou escrita.

2.4 Estrutura de Diretórios

```
chat-sockets-python/

+-- http_server.py # Servidor HTTP Flask

+-- requirements.txt # Dependências Python

+-- render.yaml # Configuração de deploy (Render)

+-- users.json # Base de dados de usuários

+-- server/

    +-- main_server.py # Servidor WebSocket primário (8765)

    +-- backup_server.py # Servidor backup + monitor (8766)

    +-- websocket.py # Handlers de conexão/mensagem

    +-- client_thread.py # Processador de mensagens por cliente

    +-- globals.py # Estado global compartilhado

    +-- room.py # Gerenciamento de salas

    \-- user.py # Autenticação e persistência

\-- client/

    +-- index.html # SPA com 3 telas (login, lobby, chat)

    +-- style.css # Tema escuro com acento esmeralda

    \-- app.js # Lógica frontend (784 linhas)
```

2.5 Protocolo de Mensagens (JSON)

Toda comunicação usa JSON com campo type como discriminador:

Tipo	Direção	Descrição
login	C → S	Autenticar usuário existente
register	C → S	Registrar novo usuário
join_room	C → S	Entrar em uma sala
message	C ↔ S	Enviar / receber mensagem

typing	$C \rightarrow S$	Indicador de digitação
room_created	$C \rightarrow S$	Criar nova sala dinamicamente
login_success	$S \rightarrow C$	Confirmação de login
room_list	$S \rightarrow C$	Lista de salas após autenticação
history	$S \rightarrow C$	Histórico: últimas 50 mensagens
online_users	$S \rightarrow C$	Lista de usuários conectados
user_joined	$S \rightarrow C$	Notificação de entrada na sala

2.6 Regras de Funcionamento

- Ao fazer login, o servidor envia imediatamente a lista de salas.
- Ao entrar em uma sala, o cliente recebe as últimas 50 mensagens de histórico.
- Mensagens são transmitidas para todos os membros da sala em tempo real.
- Usernames são únicos por sessão — login duplicado é rejeitado.
- Salas criadas dinamicamente são propagadas a todos os clientes.
- Ao desconectar, o usuário é removido de todas as salas e os demais são notificados.

3. Guia de Instalação das Dependências

3.1 Pré-requisitos

- Python 3.8 ou superior (recomendado: Python 3.11)
- pip — gerenciador de pacotes Python (incluso no Python 3.4+)
- Navegador moderno com suporte a WebSocket (Chrome, Firefox, Edge)

3.2 Passo a passo (ambiente local)

```
# 1. Clonar o repositório git clone https://github.com/matheusbuttow/chat-sockets-python # 2. Criar ambiente virtual (recomendado) python -m venv venv venv\Scripts\activate # Windows source venv/bin/activate # Linux / macOS # 3. Instalar dependências pip install -r requirements.txt # 4. Iniciar o WebSocket primário python server/main_server.py # 5. Em outro terminal: WebSocket backup python server/backup_server.py # 6. Em outro terminal: servidor HTTP python http_server.py # 7. Abrir no navegador http://localhost:5000
```

3.3 Dependências

Biblioteca	Versão	Função
websocket-server	0.6.4	Implementação do protocolo WebSocket para Python
Flask	3.1.1	Servidor HTTP para frontend estático e /api/config
gunicorn	23.0.0	Servidor WSGI para produção (usado no Render)

3.4 Deploy em produção (Render)

O arquivo render.yaml declara três serviços independentes. As URLs dos servidores WebSocket são injetadas como variáveis de ambiente (WS_PRIMARY e WS_SECONDARY) e entregues ao frontend via /api/config no startup. O mesmo build funciona em localhost e em produção sem modificação.

4. Dificuldades Enfrentadas e Soluções Adotadas

4.1 Thread safety com estado global compartilhado

Problema: O maior desafio técnico foi garantir consistência dos dados quando múltiplas threads modificam o mesmo dicionário simultaneamente. Race conditions nesse cenário são intermitentes e extremamente difíceis de reproduzir em testes.

Solução: Todos os dicionários globais passaram a ser acessados exclusivamente dentro de blocos with lock:. A regra adotada: qualquer leitura ou escrita em estado compartilhado adquire o lock correspondente, sem exceção.

4.2 Reconexão automática no frontend

Problema: Reconexão imediata causava um loop de erros quando o servidor estava indisponível: o navegador tentava centenas de vezes por segundo, inundando os logs e travando a interface.

Solução: A solução foi exponential backoff: o intervalo entre tentativas dobra a cada falha (1 s → 2 s → 4 s → ... → 30 s de máximo). Após 3 falhas no servidor primário, o frontend muda automaticamente para o backup.

4.3 Sincronização de salas criadas dinamicamente

Problema: Salas criadas por um cliente precisavam aparecer para todos os outros usuários já conectados e também para quem fizesse login depois. Inicialmente a sala existia apenas no estado local do cliente.

Solução: O evento room_created passou a ser processado pelo servidor, que registra a sala em sua lista global e retransmite o evento a todos os clientes conectados. Novos logins recebem a lista atualizada via room_list.

4.4 Deploy com três serviços e URLs dinâmicas

Problema: Configurar o Render para orquestrar três processos com URLs que se referenciavam mutuamente exigiu cuidado com ordem de inicialização e variáveis de ambiente — um serviço precisava da URL do outro para funcionar.

Solução: O endpoint /api/config centralizou essa configuração: o frontend consulta o endpoint no startup e descobre as URLs dinamicamente. No Render, basta configurar as variáveis de ambiente corretas em cada serviço.

5. Conclusão e Lições Aprendidas

O Chat Canguçu atingiu os objetivos propostos. Na prática, ficou evidente o contraste entre polling HTTP e WebSocket: enquanto o primeiro trata mensagens como requisições independentes com overhead completo de protocolo, o WebSocket reduz a entrega a um frame binário mínimo sobre uma conexão já estabelecida. A diferença de latência é perceptível mesmo em rede local.

A arquitetura de uma thread por cliente é didaticamente clara e funcionou bem para o escopo do projeto. Para produção em larga escala, a alternativa natural seria substituir threads por um event loop assíncrono com `asyncio` + `websockets`: um único processo atende milhares de conexões simultâneas sem o overhead de troca de contexto entre threads.

Lições aprendidas

Concorrência é simples de introduzir, difícil de acertar.

Race conditions em dicionários Python são silenciosas — não levantam exceção, apenas produzem resultados incorretos. A disciplina de proteger todo acesso com lock, mesmo quando parece desnecessário, provou ser essencial.

Separação de responsabilidades facilita o deploy.

Ter três processos independentes permitiu reiniciar cada componente isoladamente durante o desenvolvimento e tornou o failover natural: o backup já existia como processo separado, esperando ser necessário.

Frameworks existem por boas razões.

Gerenciar o estado da SPA em um objeto global sem React ou Vue funcionou, mas ficou claro que coordenar atualizações manuais de DOM escala mal. À medida que novos eventos foram adicionados, a lógica de renderização cresceu de forma descontrolada.

Pensar no deploy desde o início economiza retrabalho.

Adaptar o projeto para o Render depois de pronto exigiu refatorar o servidor HTTP e introduzir variáveis de ambiente — mudanças triviais se planejadas no início.

O projeto cumpriu sua função didática: tornar concreto o que a teoria descreve sobre WebSockets, concorrência e tolerância a falhas. As limitações encontradas — ausência de hashing de senhas, persistência em memória, escalabilidade limitada da abordagem threaded — são pontos de partida claros para evoluções futuras.